>_ MCMP113

Introduction to Basic Algorithms and Programming

Week 10

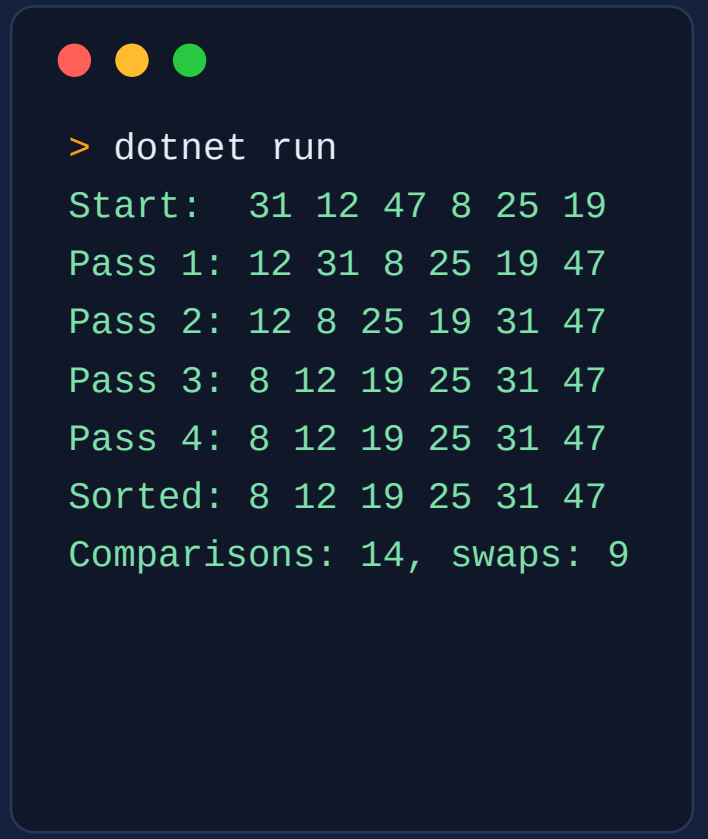
Array Operations: Searching, Sorting, Max/Min

Max/min with index · statistics · linear and binary search · bubble, selection and insertion sort · counting the work

Asst. Prof. Hasan Oğuz, PhD

Istanbul Okan University · Vocational School

2026–2027 Fall Semester



>_ LEARNING OBJECTIVES

By the end of this week you can …

Find max and min

The champion pattern with its index; second largest; avoid the "start at 0" bug.

Compute statistics

Sum, mean, variance, standard deviation; counting with conditions.

Search an array

Linear search: first index, all positions, count; −1 = not found.

Binary search

Halve a SORTED array; trace low, mid, high step by step.

Sort an array

Bubble sort (early exit), selection sort, insertion sort; swap with temp.

Count the work

Comparisons: n, $\log_2 n$, $n^2/2$; Array.Sort for real programs.

Arrays store many values — now we work with them

Traversal with an index

```
// Week 9: declare, fill, traverse
int[] scores = { 72, 85, 64, 85, 90, 78 };
int sum = 0;
for (int i = 0; i < scores.Length; i++)
{
    sum += scores[i];
}
Console.WriteLine(sum);   // 474
```

Today: the three classic array algorithms — find the extreme, search, sort.

scores (Length = 6)

72	85	64	85	90	78
0	1	2	3	4	5

index 0 … Length − 1 = 5

Where is the largest?
max = 90 at index 4

Is 64 in the array?
yes, at index 2

Put them in order
64 72 78 85 85 90



PART 1

Max, min and statistics

The champion pattern · maximum with its index · second largest · mean, variance, standard deviation

The champion pattern: the first element is the first champion

Pseudocode

```
max ← a[0]
maxIndex ← 0
FOR i ← 1 TO n − 1
  IF a[i] > max THEN
    max ← a[i]
    maxIndex ← i
  END IF
END FOR
PRINT max, maxIndex
```



min: the same loop with < instead of >.

temps (Example 1, sample run 1)

12.5	18	9.5	21	15
0	1	2	3	4

i	temps[i]	temps[i] > max ?	max	maxIndex
–	–	start	12.5	0
1	18	true	18	1
2	9.5	false	18	1
3	21	true	21	3
4	15	false	21	3

Result: max = 21 at index 3 (day 4).

>_ EXAMPLE 1 · 10_01_MAXMININDEX

Warmest and coldest day in one loop

Program.cs (excerpt) · then print max, maxIndex + 1, min, minIndex + 1

```
double max = temps[0];        // first element = first candidate
double min = temps[0];
int maxIndex = 0;
int minIndex = 0;
for (int i = 1; i < n; i++)  // temps[0] is already used
{
    if (temps[i] > max)
    {
        max = temps[i];
        maxIndex = i;
    }
    if (temps[i] < min)
    {
        min = temps[i];
        minIndex = i;
    }
}
```

Console

```
How many days? 5
Temperature of day 1: 12.5
Temperature of day 2: 18
Temperature of day 3: 9.5
Temperature of day 4: 21
Temperature of day 5: 15
Warmest: 21 on day 4 (index 3)
Coldest: 9.5 on day 3 (index 2)
Max if we start at 0: 21
```



Value AND position. Keep the index; the value is always temps[maxIndex].

>_ CLASSIC BUG

Why not start the maximum at 0?

Program.cs (end)

```
// The classic mistake: starting the maximum at 0
double wrongMax = 0;
for (int i = 0; i < n; i++)
{
    if (temps[i] > wrongMax)
    {
        wrongMax = temps[i];
    }
}
Console.WriteLine($"Max if we start at 0: {wrongMax}");
```

Console · a cold week

```
How many days? 4
Temperature of day 1: -3.5
Temperature of day 2: -1
Temperature of day 3: -7.25
Temperature of day 4: -2
Warmest: -1 on day 2 (index 1)
Coldest: -7.25 on day 3 (index 2)
Max if we start at 0: 0
```

Bug: all values are below 0, so none is "> 0". The answer 0 is not even in the array!

Fix: max ← a[0] (and loop from 1), or max ← double.MinValue. Always test with negative data.

Second largest in one pass: two champions

Program.cs (excerpt)

```
int largest = scores[0];
int second = 0;
bool hasSecond = false;
for (int i = 1; i < scores.Length; i++)
{
    if (scores[i] > largest)
    {
        second = largest;
        hasSecond = true;
        largest = scores[i];
    }
    else if (scores[i] < largest && (!hasSecond || scores[i] > second))
    {
        second = scores[i];
        hasSecond = true;
    }
    // scores[i] == largest: a duplicate, ignore it
}
```

i	scores[i]	case	largest	second
–	72	start	72	–
1	85	new largest	85	72
2	64	too small	85	72
3	85	duplicate	85	72
4	90	new largest	90	85
5	78	too small	90	85

Console

```
Scores: 72, 85, 64, 85, 90, 78
Largest: 90
Second largest: 85
```

Mean, variance and standard deviation

mean

$$\bar{x} = (x_1 + x_2 + \ldots + x_n) / n$$

variance

$$\sigma^2 = \Sigma (x_i - \bar{x})^2 / n$$

standard deviation

$$\sigma = \sqrt{\sigma^2} \quad \text{(Math.Sqrt)}$$

x_i	$x_i - \bar{x}$	$(x_i - \bar{x})^2$
72	5.67	32.11
45	-21.33	455.11
88	21.67	469.44
60	-6.33	40.11
95	28.67	821.78
38	-28.33	802.78
Σ = 398	**$\bar{x}$ = 66.33**	**2621.33**

Two passes: first the mean, then the squared distances from it.

$$\sigma^2 = 2621.33 / 6 = 436.89$$
$$\sigma = \sqrt{436.89} \approx 20.90 \text{ points}$$

Grade statistics: two passes and counting

pass 2: spread

```
double squares = 0;
for (int i = 0; i < n; i++)
{
    double d = grades[i] - mean;
    squares += d * d;
}
double variance = squares / n;
double sigma = Math.Sqrt(variance);
```

$$\sigma^2 = \Sigma\ (x_i - \bar{x})^2\ /\ n$$

squares / n, then Math.Sqrt

pass 3: counting

```
int passed = 0;
int aboveMean = 0;
for (int i = 0; i < n; i++)
{
    if (grades[i] >= 50)
    {
        passed++;
    }
    if (grades[i] > mean)
    {
        aboveMean++;
    }
}
```

Console

```
Number of students: 6
Grade 1: 72
Grade 2: 45
Grade 3: 88
Grade 4: 60
Grade 5: 95
Grade 6: 38
Sum      = 398
Mean     = 66.33
Variance = 436.89
Std dev  = 20.90
Passed (>= 50): 4 of 6
Above the mean: 3 of 6
```

count++ only inside the if.



PART 2

Searching

Linear search: one by one · binary search: halve a sorted array · −1 means "not found"

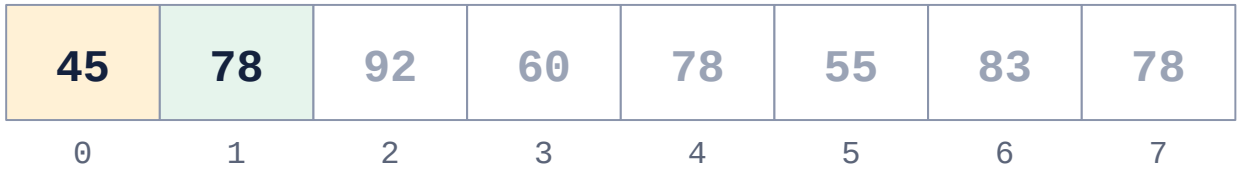
Look at the elements one by one

First occurrence

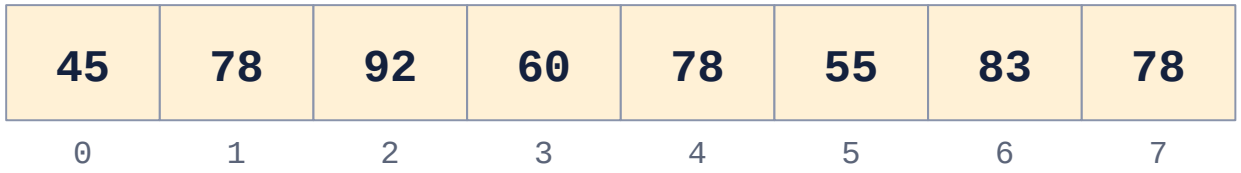
```
found ← false
index ← -1
i ← 0
WHILE i < n AND NOT found
  IF a[i] = target THEN
    found ← true
    index ← i
  END IF
  i ← i + 1
END WHILE
```

🏁 **-1 = "not found"**

Search 78: found at index 1 after 2 checks



Search 50: not found after 8 checks



checked, no match match never looked at

Best case 1 check, worst case n checks. Works on ANY array, sorted or not.

First index, all positions and the count

1) first occurrence

```
int index = -1;
bool found = false;
int comparisons = 0;
int i = 0;
while (i < scores.Length && !found)
{
    comparisons++;
    if (scores[i] == target)
    {
        found = true;
        index = i;
    }
    i++;
}
```

2) all positions and the count

```
int count = 0;
Console.Write("All positions:");
for (int k = 0; k < scores.Length; k++)
{
    if (scores[k] == target)
    {
        Console.Write($" {k}");
        count++;
    }
}
Console.WriteLine();
Console.WriteLine($"Count: {count}");
```

Search 78

```
Search for: 78
First at index 1: 2 checks
All positions: 1 4 7
Count: 3
```

Search 50

```
Search for: 50
Not found: 8 checks
All positions:
Count: 0
```

Guess the number again: look in the middle

Binary search (a must be sorted)

```
low ← 0
high ← n − 1
index ← −1
WHILE low ≤ high AND index = −1
  mid ← (low + high) div 2
  IF a[mid] = target THEN
    index ← mid
  ELSE IF a[mid] < target THEN
    low ← mid + 1      // go right
  ELSE
    high ← mid − 1     // go left
  END IF
END WHILE
```



Week 1 / Week 7 game

Secret in 1..100: guess 50, "higher" → 51..100, guess 75 … at most 7 guesses.



Requirement: sorted

Only then does a[mid] < target prove that the target is to the RIGHT of mid.



Half of the range each step

15 → 7 → 3 → 1 elements: at most 4 steps for 15 values, 20 steps for one million.

Searching 20 in 15 sorted values: low, mid, high



After step 4: high = 3 < low = 4 → empty range → "Not found (low > high), steps: 4"

Binary search in C# with a printed trace

```
inside while (low <= high && index == -1)
int mid = (low + high) / 2;          // integer division
step++;
Console.Write($"Step {step}: low={low} high={high} ");
Console.WriteLine($"mid={mid} a[mid]={a[mid]}");
if (a[mid] == target)
{
    index = mid;
}
else if (a[mid] < target)
{
    low = mid + 1;                   // go right
}
else
{
    high = mid - 1;                  // go left
}
```

Search 58

```
Search for: 58
Step 1: low=0 high=14 mid=7 a[mid]=36
Step 2: low=8 high=14 mid=11 a[mid]=58
Found at index 11, steps: 2
```

Search 77

```
Search for: 77
Step 1: low=0 high=14 mid=7 a[mid]=36
Step 2: low=8 high=14 mid=11 a[mid]=58
Step 3: low=12 high=14 mid=13 a[mid]=70
Step 4: low=14 high=14 mid=14 a[mid]=77
Found at index 14, steps: 4
```

```
a = 4 9 13 18 22 27 31 36 40 45 51 58 63 70 77
```

How many comparisons for a value that is not there?

n	linear search	binary search	$\lfloor \log_2 n \rfloor + 1$
10	10	4	4
100	100	7	7
1000	1000	10	10
10000	10000	14	14
1 000 000	1 000 000	20	20

Rows 1–4: real output of 10_13_SortingRace; last row: formula.

Linear: O(n)

Work grows like n. Any array.

Binary: O(log n)

Doubling n adds one step. Sorted arrays only.



PART 3

Sorting

Swapping two cells · bubble sort with early exit · selection sort · insertion sort · counting comparisons

Exchange a[i] and a[j]: you need a third box

① `temp ← a[0]`



② `a[0] ← a[1]`



③ `a[1] ← temp`



C#

```
int temp = a[i];   // swap via temp
a[i] = a[j];
a[j] = temp;

(a[i], a[j]) = (a[j], a[i]);   // tuple swap
```

Without temp

```
// WRONG: 31 is lost
a[i] = a[j];   // 12 12
a[j] = a[i];   // 12 12
```

Compare neighbours, swap if they are in the wrong order

Bubble sort with early exit

```
FOR pass ← 1 TO n − 1
  swapped ← false
  FOR j ← 0 TO n − pass − 1
    IF a[j] > a[j + 1] THEN
      swap a[j] and a[j + 1]
      swapped ← true
    END IF
  END FOR
  IF NOT swapped THEN
    stop: the array is sorted
  END IF
END FOR
```



Neighbours only

Each comparison looks at a[j] and a[j + 1].



Largest goes last

After pass p, the last p cells are final.



Early exit

A pass without swaps proves that the array is sorted.

Pass 1: 5 comparisons, 4 swaps — 47 bubbles to the end

j = 0	31	12	47	8	25	19	31 > 12 ? yes → swap
j = 1	12	31	47	8	25	19	31 > 47 ? no → keep
j = 2	12	31	47	8	25	19	47 > 8 ? yes → swap
j = 3	12	31	8	47	25	19	47 > 25 ? yes → swap
j = 4	12	31	8	25	47	19	47 > 19 ? yes → swap
after	12	31	8	25	19	47	Pass 1: 12 31 8 25 19 47

The sorted tail grows from the right

Start	**31**	**12**	**47**	**8**	**25**	**19**	
Pass 1	**12**	**31**	**8**	**25**	**19**	**47**	**5 comparisons, 4 swaps**
Pass 2	**12**	**8**	**25**	**19**	**31**	**47**	**4 comparisons, 3 swaps**
Pass 3	**8**	**12**	**19**	**25**	**31**	**47**	**3 comparisons, 2 swaps**
Pass 4	**8**	**12**	**19**	**25**	**31**	**47**	**2 comparisons, 0 swaps → stop!**



Total: 14 comparisons, 9 swaps. Pass 4 finds no swap, so pass 5 is skipped.

Bubble sort in C#: two nested loops and a flag

Program.cs (excerpt) · passes and comparisons

```
for (int pass = 1; pass <= n - 1; pass++)
{
    bool swapped = false;
    for (int j = 0; j < n - pass; j++)
    {
        comparisons++;
        if (a[j] > a[j + 1])
        {
            int temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
            swapped = true;
            swaps++;
        }
    }
```

… end of each pass

```
Console.WriteLine($"Pass {pass}: " + string.Join(" ", a));
if (!swapped)
{
    break;
}
}
```

Console

```
Start:  31 12 47 8 25 19
Pass 1: 12 31 8 25 19 47
Pass 2: 12 8 25 19 31 47
Pass 3: 8 12 19 25 31 47
Pass 4: 8 12 19 25 31 47
Sorted: 8 12 19 25 31 47
Comparisons: 14, swaps: 9
```

Select the smallest and put it in front: one swap per pass

Selection sort

```
FOR i ← 0 TO n − 2
  minIndex ← i
  FOR j ← i + 1 TO n − 1
    IF a[j] < a[minIndex] THEN
      minIndex ← j
    END IF
  END FOR
  IF minIndex ≠ i THEN
    swap a[i] and a[minIndex]
  END IF
END FOR
```

Each row: array BEFORE the swap of pass i

i = 0	31	12	47	8	25	19	31 ↔ 8
i = 1	8	12	47	31	25	19	no swap
i = 2	8	12	47	31	25	19	47 ↔ 19
i = 3	8	12	19	31	25	47	31 ↔ 25
i = 4	8	12	19	25	31	47	no swap

minimum · position i · final

Inner loop = "minimum with index" from Part 1.

Selection sort: 15 comparisons, only 3 swaps

Program.cs (excerpt)

```
for (int i = 0; i < n - 1; i++)
{
    int minIndex = i;          // where the smallest is
    for (int j = i + 1; j < n; j++)
    {
        comparisons++;
        if (a[j] < a[minIndex])
        {
            minIndex = j;
        }
    }
    if (minIndex != i)         // at most one swap per pass
    {
        int temp = a[i];
        a[i] = a[minIndex];
        a[minIndex] = temp;
        swaps++;
    }
```

Console

```
Start:  31 12 47 8 25 19
Pass 1: 8 12 47 31 25 19   (min = 8)
Pass 2: 8 12 47 31 25 19   (min = 12)
Pass 3: 8 12 19 31 25 47   (min = 19)
Pass 4: 8 12 19 25 31 47   (min = 25)
Pass 5: 8 12 19 25 31 47   (min = 31)
Sorted: 8 12 19 25 31 47
Comparisons: 15, swaps: 3
```

	bubble	selection
comparisons	14	15
swaps	9	3

Insertion sort: like sorting playing cards

Program.cs (excerpt)

```
for (int i = 1; i < n; i++)
{
    int key = a[i];            // the "card" to insert
    int j = i - 1;
    while (j >= 0)
    {
        comparisons++;
        if (a[j] > key)
        {
            a[j + 1] = a[j];   // shift one cell right
            shifts++;
            j--;
        }
        else
        {
            break;             // found the place
        }
    }
    a[j + 1] = key;
```

Console

```
Start:  31 12 47 8 25 19
Pass 1: 12 31 47 8 25 19   (12 -> [0])
Pass 2: 12 31 47 8 25 19   (47 -> [2])
Pass 3: 8 12 31 47 25 19   (8 -> [0])
Pass 4: 8 12 25 31 47 19   (25 -> [2])
Pass 5: 8 12 19 25 31 47   (19 -> [2])
Sorted: 8 12 19 25 31 47
Comparisons: 12, shifts: 9
```

Sorted left part

Pass i inserts a[i] into the sorted part a[0..i−1]: shift bigger values right, then a[j + 1] = key.

Counting the work for growing n

Console

```
     n  linear  binary  bubble-cmp  bubble-swp  select-cmp  select-swp  bubble-ms
    10      10       4          42          24          45           8          0
   100     100       7        4935        2162        4950          94          0
  1000    1000      10      497484      244419      499500         991          2
 10000   10000      14    49994565    25154921    49995000        9989        290
Worst case of both sorts: n(n-1)/2 comparisons.
```

O(n) · linear

n = 10 000 → 10 000 checks.

O(log n) · binary

n = 10 000 → 14 checks.

$O(n^2)$ · sorting

n(n − 1)/2 = 49 995 000 comparisons. ms column varies per PC.



PART 4

Built-ins and more array algorithms

Array.Sort · IndexOf · BinarySearch · Reverse · reverse in place · rotate · remove duplicates · merge

What professionals use: the Array class

Program.cs (excerpt)

```
Console.WriteLine($"IndexOf(25):   {Array.IndexOf(a, 25)}");
Console.WriteLine($"IndexOf(99):   {Array.IndexOf(a, 99)}");

Array.Sort(a);
Console.WriteLine("After Sort:    " + string.Join(" ", a));

Console.WriteLine($"BinarySearch(25): {Array.BinarySearch(a, 25)}");
int pos = Array.BinarySearch(a, 20);
Console.WriteLine($"BinarySearch(20): {pos}  (insert at {-pos - 1})");

Array.Reverse(a);
Console.WriteLine("After Reverse: " + string.Join(" ", a));

string[] names = { "Zeynep", "ali", "Can", "Deniz", "Emre" };
Array.Sort(names);
Console.WriteLine("Ordinal:     " + string.Join(", ", names));
Array.Sort(names, StringComparer.OrdinalIgnoreCase);
Console.WriteLine("Ignore case: " + string.Join(", ", names));
```

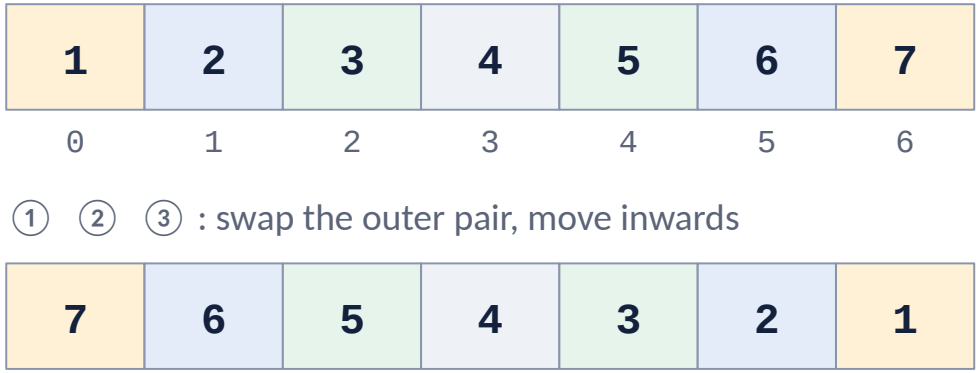
Console

```
Original:      31 12 47 8 25 19
IndexOf(25):   4
IndexOf(99):   -1
After Sort:    8 12 19 25 31 47
BinarySearch(25): 3
BinarySearch(20): -4  (insert at 3)
After Reverse: 47 31 25 19 12 8
Ordinal:     Can, Deniz, Emre, Zeynep, ali
Ignore case: ali, Can, Deniz, Emre, Zeynep
```



BinarySearch only on sorted arrays. No LINQ: no .Max(), .OrderBy().

Reverse in place, rotate with %



reverse in place

```
while (left < right)
{
    int temp = a[left];
    a[left] = a[right];
    a[right] = temp;
    left++;
    right--;
}
```

rotate left by k

```
int[] rotated = new int[n];
for (int i = 0; i < n; i++)
{
    rotated[(i - k + n) % n] = b[i];
}
```

Console · two runs (k = 3, k = 9)

```
Original:       1 2 3 4 5 6 7
Reversed:       7 6 5 4 3 2 1
Rotate left by k = 3
Rotated left:   4 5 6 7 1 2 3
Rotate left by k = 9
Rotated left:   3 4 5 6 7 1 2
```

Remove duplicates from a sorted array

before

2	2	3	5	5	5	8	9	9
0	1	2	3	4	5	6	7	8

after the loop (unique = 5)

2	3	5	8	9	5	8	9	9
0	1	2	3	4	5	6	7	8

kept (first of its value) ignored

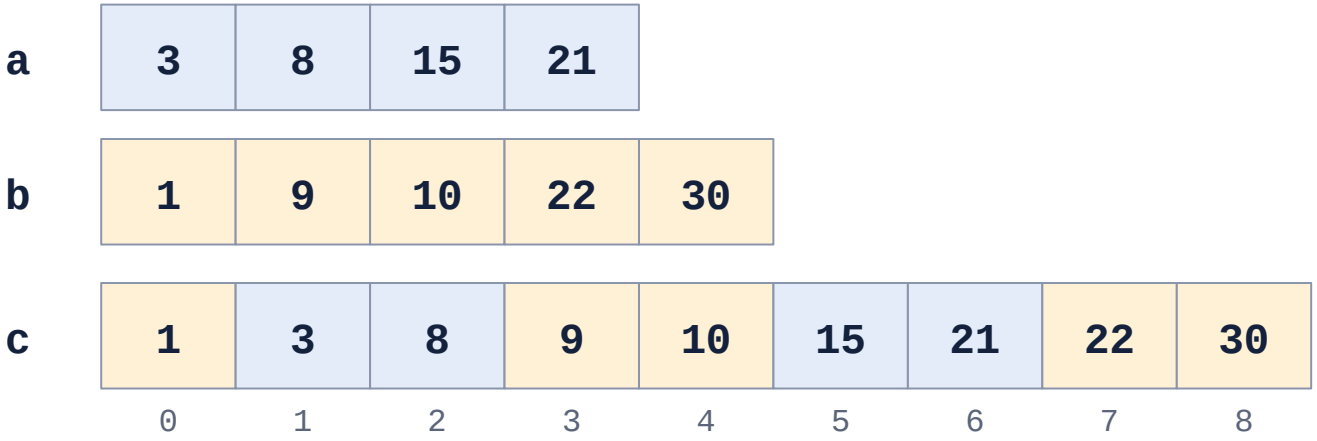
Console

```
Sorted input: 2 2 3 5 5 5 8 9 9
Unique (5): 2 3 5 8 9
Removed duplicates: 4
```

Program.cs (excerpt)

```
int unique = 1;
for (int i = 1; i < a.Length; i++)
{
    if (a[i] != a[unique - 1])
    {
        a[unique] = a[i];
        unique++;
    }
}
```

Merge two sorted arrays



Console

```
a: 3 8 15 21
b: 1 9 10 22 30
merged: 1 3 8 9 10 15 21 22 30
```

Colour = source array. 9 elements, at most 8 comparisons.

```
main loop
while (i < a.Length && j < b.Length)
{
    if (a[i] <= b[j])
    {
        c[k] = a[i];
        i++;
    }
    else
    {
        c[k] = b[j];
        j++;
    }
    k++;
}
```

Eight array-algorithm bugs and how to fix them

Mistake	Symptom	Fix
`max = 0;`	wrong max for negative data	max = a[0]; loop from 1
`j < n  with  a[j + 1]`	IndexOutOfRangeException	j < n – pass (or n – 1)
swap without temp	one value lost (12 12)	temp, or tuple swap
binary search on unsorted data	wrong "not found"	sort first (Array.Sort)
`low = mid; / high = mid;`	infinite loop	mid + 1 and mid – 1
return value 0 for "not found"	0 is a valid index!	use −1
reverse loop over all i	array unchanged	stop when left meets right
sort the original array by accident	original order lost	sort a copy: (int[])a.Clone()

Quiz: what is printed?

Q1 · minimum

```
int[] a = { 5, 3, 9, 1, 7 };
int m = 0;
for (int i = 1; i < a.Length; i++)
{
    if (a[i] < a[m])
    {
        m = i;
    }
}
Console.WriteLine($"{m} {a[m]}");
```

Q2 · binary search for 23

```
int[] b = { 2, 5, 8, 12, 16,
            23, 38, 56, 72, 91 };
int low = 0;
int high = b.Length - 1;
while (low <= high)
{
    int mid = (low + high) / 2;
    Console.Write($"{mid} ");
    if (b[mid] == 23)
    {
        break;
    }
    else if (b[mid] < 23)
    {
        low = mid + 1;
    }
    else
    {
        high = mid - 1;
    }
}
```

Q3 · one bubble pass

```
int[] c = { 4, 2, 5, 1, 3 };
int swaps = 0;
for (int j = 0; j < c.Length - 1; j++)
{
    if (c[j] > c[j + 1])
    {
        int t = c[j];
        c[j] = c[j + 1];
        c[j + 1] = t;
        swaps++;
    }
}
string line = string.Join(" ", c);
Console.WriteLine($"{line} | {swaps}");
```

Answers (checked by running the code)

Q1

```
3 1
```

i	a[i]	m
–	–	0
1	3	1
2	9	1
3	1	3
4	7	3

Q2

```
4 7 5
```

low	high	mid	b[mid]
0	9	4	16
5	9	7	56
5	6	5	23 ✓

Q3

```
2 4 1 3 5 | 3
```

start	4	2	5	1	3
j=0	2	4	5	1	3
j=1	2	4	5	1	3
j=2	2	4	1	5	3
j=3	2	4	1	3	5



Q2: 3 comparisons instead of 6 (linear). Q3: one pass moves the largest value to the end.

Your turn (pairs, 25 minutes)

E1 · Range of the week ★

Read 7 temperatures. Print max − min (the range) and the day numbers (1–7) of the max and the min.

E2 · Find a student ★

string[] names = {...}. Read a name, print its index or "not found" (−1). Ignore upper/lower case.

E3 · Descending order ★★

Change bubble sort (Example 6) so that it sorts from largest to smallest. Count passes, comparisons and swaps.

E4 · Every target ★★

Binary search each of the 15 values of Example 5 in turn. Print the steps for each and the average.

>_ HOMEWORK

Before next week

H1 · Median ★★

Read n grades, sort a COPY (keep the original), print the original order, the sorted order and the median (n even or odd).

H2 · Two smallest ★

Print the smallest and the second smallest DIFFERENT value of an array, or a message if it does not exist.

H3 · Mode ★★

Sort an int array, then find the most frequent value in one pass (count runs of equal neighbours).

H4 · Is it sorted? ★

Check with one loop whether an array is sorted ascending; print the first index where the order breaks.

Submit the four Program.cs files on the course group; include one console run for each.

>_ SUMMARY

Key Takeaways

01 Max/min: start with a[0], loop from 1, keep the index. Test with negative data.

02 Statistics need the array: mean in pass 1, squared distances in pass 2, σ = √variance.

03 Linear search: any array, up to n checks; −1 means "not found".

04 Binary search: SORTED arrays only; low/mid/high; about $\log_2 n$ steps.

05 Bubble sort swaps neighbours (early exit); selection sort selects the minimum (≤ n − 1 swaps).

06 Simple sorts make about $n^2/2$ comparisons; in real programs call Array.Sort.

NEXT WEEK

Week 11: Multi-dimensional arrays (matrices)

>_

MCMP113
Course materials